

PREDICCIÓN DE ACCIDENTE CEREBROVASCULAR

Stroke Prediction Dataset - Análisis con Machine Learning

APRENDIZAJE DE MÁQUINAS (ACIF104)

Informe Fase 2 - Trabajo Evaluado (Mejora de acuerdo a observaciones)

Priscila Arganaraz | Carlos González

Grupo 04 - ACIF104 Aprendizaje de Maquinas

Ingeniería Civil Informática - Universidad Andrés Bello

Mayo 2026

GitHub: https://github.com/carlosrgv1975/acif104_s6_grupo04_Arganaraz_Gonzalez

INDICE

1. INTRODUCCIÓN Y OBJETIVOS	3
1.1 Objetivos Específicos	3
2. PROBLEMÁTICA	3
2.1 Contexto y relevancia del problema	3
2.2 Análisis crítico de trabajos previos	4
2.3 Justificación organizacional y de impacto.....	5
2.4 Requisitos iniciales del proyecto.....	6
3. METODOLOGÍA.....	7
3.1 Descripción general.....	7
3.2 Fases metodológicas.....	7
3.3 Plan de trabajo	8
3.4 Distribución de responsabilidades	9
3.5 Herramientas y entorno tecnológico	9
4. ANÁLISIS EXPLORATORIO DE DATOS (EDA)	10
4.1 Distribución de Variables Numéricas.....	10
4.2 Matriz de Correlación y Distribución por Clase.....	11
4.3 Variables Categóricas vs Stroke.....	11
4.4 Requisitos del Proyecto	12
5. COMPARACIÓN DE MODELOS DE MACHINE LEARNING	14
5.1 Resultados por Modelo.....	14
5.2 Reportes de Clasificación	15
6. TÉCNICAS DE BALANCEO DE CLASES.....	16
7. RED NEURONAL CON KERAS — COMPARACIÓN DE 3 ARQUITECTURAS.....	17
7.1 Justificación del enfoque de Deep Learning	17
7.2 Arquitectura 1 — MLP Base (64→32→16→1)	17
7.3 Arquitectura 2 — MLP con Regularización L2 (64→32→16→1)	18
7.4 Arquitectura 3 — MLP Profundo (128→64→32→16→1)	19
7.5 Tabla comparativa de arquitecturas Deep Learning	19
7.6 Análisis y selección de arquitectura.....	20
8. COMPARACIÓN FINAL DE TODOS LOS MODELOS.....	21
9. EXPLICABILIDAD DEL MODELO CON SHAP	22
9.1 Importancia Global de Variables.....	22
9.2 Impacto por Variable (Beeswarm)	23
9.3 Conclusiones de Explicabilidad	23
10. APLICACIÓN WEB — STREAMLIT	24
10.1 Características.....	24
10.2 Ejemplo de Predicción	24



11. CONCLUSIONES	25
11.1 Trabajo Futuro	25
12. REFERENCIAS	26

1. INTRODUCCIÓN Y OBJETIVOS

Este trabajo corresponde al proyecto final del curso ACIF104 Aprendizaje de Maquinas de la Universidad Andrés Bello. El objetivo principal es desarrollar un sistema de predicción de accidente cerebrovascular (stroke) utilizando técnicas de Machine Learning, con especial atención al manejo del desbalance de clases, la explicabilidad del modelo y el despliegue en una aplicación web interactiva.

El dataset utilizado contiene información clínica y demográfica de pacientes. La variable objetivo (stroke) presenta un severo desbalance: aproximadamente el 95% de los casos corresponden a la clase negativa (sin stroke) y solo el 5% a la clase positiva.

1.1 Objetivos Específicos

- Realizar un Análisis Exploratorio de Datos (EDA) completo con visualizaciones estadísticas
- Comparar al menos 3 algoritmos de Machine Learning clásicos
- Aplicar y comparar técnicas de balanceo de clases
- Implementar una Red Neuronal con Keras
- Generar explicaciones del modelo con SHAP
- Desplegar una aplicación interactiva con Streamlit para predicción en tiempo real

2. PROBLEMÁTICA

2.1 Contexto y relevancia del problema

El accidente cerebrovascular (ACV o stroke) constituye una de las emergencias médicas más devastadoras a nivel mundial. Según la Organización Mundial de la Salud (OMS, 2024), en 2021 se registraron 11,9 millones de nuevos casos de stroke, con aproximadamente 7 millones de muertes anuales, posicionándolo como la segunda causa de muerte y la tercera causa combinada de muerte y discapacidad a nivel global, medida en años de vida ajustados por discapacidad (DALYs). El costo económico global asociado supera los US\$ 890 mil millones anuales, equivalente al 0,66% del PIB mundial, y se proyecta que se duplique hacia 2050 (Feigin et al., 2025).

La tendencia epidemiológica es alarmante: entre 1990 y 2021, los casos incidentes de stroke aumentaron un 70%, las muertes un 44% y la prevalencia un 86% (Feigin et al., 2025). Las proyecciones para 2050 indican que las muertes por stroke alcanzarán los 9,7 millones anuales, con el 91% concentrado en países de ingresos bajos y medios. Este escenario subraya la urgencia de desarrollar herramientas de detección temprana que permitan intervenciones preventivas oportunas, especialmente en sistemas de salud con recursos limitados.

El dataset utilizado en este proyecto — Healthcare Stroke Dataset de Kaggle — contiene registros clínicos y demográficos de 5.110 pacientes, incluyendo variables como edad, hipertensión, nivel de glucosa, IMC, género, tipo de trabajo y hábitos de vida. La variable objetivo binaria (stroke:



1/0) presenta un severo desbalance de clases: aproximadamente el 95% de los registros corresponde a la clase negativa y solo el 5% a la positiva. Este desbalance refleja fielmente la realidad clínica del problema y constituye el principal desafío técnico del proyecto.

2.2 Análisis crítico de trabajos previos

A continuación se presenta un análisis crítico de cinco trabajos recientes directamente relacionados con la predicción de stroke mediante aprendizaje automático:

Fuente 1 — Fernandes et al. (2024) Fernandes, J. N. D., Cardoso, V. E. M., Comesaña-Campos, A., & Pinheira, A. (2024). Comprehensive review: Machine and deep learning in brain stroke diagnosis. *Sensors*, 24(13), 4355. <https://doi.org/10.3390/s24134355>

Este trabajo de revisión sistemática, conducido bajo directrices PRISMA, analiza 25 artículos publicados entre 2020 y 2024 sobre aplicaciones de ML y DL en diagnóstico de stroke. Su contribución central es demostrar que los modelos de aprendizaje profundo superan sistemáticamente a los enfoques estadísticos tradicionales en tareas de clasificación, segmentación y detección, especialmente cuando se dispone de datos multimodales (imágenes, registros clínicos y señales fisiológicas). Una limitación crítica que señala el estudio es la escasez de benchmarks estandarizados entre trabajos, lo que dificulta la comparación directa de resultados. Para el presente proyecto, este trabajo justifica la incorporación de una red neuronal como modelo comparativo, aunque también evidencia que para datasets de tamaño moderado (como el utilizado en este proyecto), los modelos clásicos de ML pueden ser igualmente competitivos.

Fuente 2 — Mainali et al. (2021) Mainali, S., Darsie, M. E., & Smetana, K. S. (2021). Machine learning in action: Stroke diagnosis and outcome prediction. *Frontiers in Neurology*, 12, 734345. <https://doi.org/10.3389/fneur.2021.734345>

Esta revisión, publicada en *Frontiers in Neurology*, examina la aplicación clínica de algoritmos de ML ya integrados comercialmente en el diagnóstico de stroke. Los autores identifican como hallazgo central que, a pesar del avance técnico, la naturaleza de "caja negra" de muchos modelos de DL ha limitado su adopción clínica, ya que los profesionales de salud requieren outputs interpretables para la toma de decisiones. Esto justifica directamente la incorporación de SHAP en el presente proyecto como mecanismo de explicabilidad: no basta con obtener buenas métricas, sino que el modelo debe ser interpretable para ser clínicamente útil. Una limitación del trabajo es que se centra principalmente en imágenes médicas, dejando de lado los datasets de variables clínicas estructuradas, que son más accesibles y escalables en contextos de atención primaria.

Fuente 3 — Munsell et al. (2025) / Frontiers in Neurology Evaluating machine learning models for stroke prediction based on clinical variables. *Frontiers in Neurology* (2025). <https://www.frontiersin.org/journals/neurology/articles/10.3389/fneur.2025.1668420>

Este estudio compara modelos de regresión logística, SVM, random forest, gradient boosting y redes neuronales para predicción de stroke a partir de variables clínicas estructuradas. Sus hallazgos

son directamente comparables con los de este proyecto: la regresión logística con ajuste de pesos de clase obtiene resultados competitivos en recall de la clase positiva, mientras que los modelos tipo ensemble (Random Forest, XGBoost) tienden a sobreajustarse a la clase mayoritaria en presencia de desbalance severo. El trabajo también señala que la métrica AUC-ROC es más informativa que la accuracy en escenarios desbalanceados, lo que coincide con la estrategia de evaluación adoptada en este proyecto. Una limitación es que no compara técnicas de balanceo de clases entre sí.

Fuente 4 — He et al. (2025) He, J. et al. (2025). A comprehensive analysis of stroke risk factors and development of a predictive model using machine learning approaches. *Molecular Genetics and Genomics*. <https://doi.org/10.1007/s00438-024-02217-3>

Utilizando datos del China Health and Retirement Longitudinal Study (2011–2020), este trabajo identifica y jerarquiza factores de riesgo modificables y no modificables para stroke mediante ML. Sus hallazgos confirman que la edad, la hipertensión, la glucosa en ayuno, el BMI y el historial familiar son los predictores más consistentes, lo que es coherente con los resultados del análisis SHAP del presente proyecto (donde la edad domina ampliamente la predicción con un valor SHAP de ~1.6). Una limitación relevante del estudio es que utiliza datos longitudinales de seguimiento, lo que le otorga mayor poder predictivo pero también mayor complejidad de replicación que datasets transversales como el empleado aquí. Su aporte al presente trabajo es validar externamente los factores de riesgo identificados por SHAP.

Fuente 5 — Comparative Analysis of Stroke Prediction Models (2025) Comparative analysis of stroke prediction models using machine learning. *arXiv*, 2505.09812. <https://arxiv.org/pdf/2505.09812>

Este trabajo, que utiliza el mismo dataset de Kaggle empleado en el presente proyecto (5.110 registros), evalúa sistemáticamente cinco algoritmos (Logistic Regression, Random Forest, Decision Tree, SVM, XGBoost) comparando técnicas de sobremuestreo, submuestreo y SMOTE para mitigar el desbalance de clases. Sus resultados son directamente comparables: XGBoost obtiene el mejor balance precisión-recall de forma global, pero la regresión logística con `class_weight` destaca por su simplicidad e interpretabilidad. El análisis de importancia de características confirma que glucosa, BMI y edad son los predictores dominantes. La principal limitación del trabajo es que no incorpora modelos de deep learning ni técnicas de explicabilidad como SHAP. Este antecedente justifica directamente la decisión de ampliar el análisis comparativo del presente proyecto incorporando una red neuronal y el análisis SHAP.

2.3 Justificación organizacional y de impacto

Desde una perspectiva organizacional, los sistemas de salud enfrentan un desafío creciente: la detección temprana del riesgo de stroke demanda recursos humanos especializados que escasean en contextos de atención primaria. Una herramienta de ML capaz de evaluar el riesgo a partir de variables clínicas de rutina — sin necesidad de imágenes médicas costosas — representa una solución escalable y de bajo costo. La aplicación Streamlit desarrollada en este proyecto materializa esta visión: permite a un profesional de salud ingresar los datos del paciente en segundos y obtener

una estimación de riesgo con explicación de los factores determinantes, directamente accionable en una consulta médica.

Los trabajos revisados coinciden en tres limitaciones recurrentes que este proyecto busca abordar directamente: (1) el desbalance de clases frecuentemente no es tratado con rigor comparativo; (2) la explicabilidad del modelo rara vez se implementa; y (3) los modelos raramente se despliegan en interfaces accesibles para usuarios clínicos. Este proyecto responde a las tres mediante técnicas de balanceo comparadas, SHAP y una aplicación web interactiva.

2.4 Requisitos iniciales del proyecto

A partir del análisis del problema y de los antecedentes revisados, se identificaron los siguientes requisitos iniciales:

- **Funcionales:** predecir la probabilidad de stroke a partir de variables clínicas y demográficas disponibles en atención primaria; comparar múltiples modelos ML y técnicas de balanceo; proporcionar explicaciones interpretables de las predicciones.
- **No funcionales:** usabilidad (interfaz accesible para profesionales sin conocimientos de ML); explicabilidad (SHAP para interpretación de predicciones individuales); escalabilidad (modelo serializable y desplegable en entornos web); confiabilidad (evaluación sobre datos no vistos en entrenamiento).

3. METODOLOGÍA

3.1 Descripción general

El desarrollo de este proyecto siguió una metodología estructurada en seis fases secuenciales, basada en el ciclo estándar de proyectos de aprendizaje automático (CRISP-DM adaptado): comprensión del problema, análisis exploratorio de datos, selección y comparación de técnicas, elaboración de modelos, evaluación e interpretación, y despliegue. Cada fase fue documentada en un Jupyter Notebook (.ipynb) versionado en el repositorio GitHub del proyecto.

3.2 Fases metodológicas

Fase 1 — Comprensión del problema y análisis bibliográfico Se definió la problemática clínica, se identificaron los requisitos del proyecto y se revisaron al menos cinco trabajos previos recientes relacionados con predicción de stroke mediante ML. Esta fase establece el marco teórico que justifica las decisiones técnicas posteriores.

Fase 2 — Análisis Exploratorio de Datos (EDA) Se analizaron las 11 variables del dataset (3 numéricas, 8 categóricas/binarias), evaluando completitud (valores nulos en BMI), consistencia, correctitud y presencia de outliers mediante boxplots. Se generaron estadísticas descriptivas, histogramas, matriz de correlación y gráficos de distribución de variables por clase objetivo, identificando patrones relevantes como la concentración de casos positivos en pacientes mayores de 40 años.

Fase 3 — Selección y comparación de técnicas candidatas Se propusieron y justificaron cuatro técnicas de aprendizaje automático: Random Forest, Regresión Logística, XGBoost (modelos clásicos supervisados) y una Red Neuronal con Keras (modelo de aprendizaje profundo). La selección se fundamentó en el análisis crítico de trabajos previos y en las características del problema (clasificación binaria, desbalance de clases, variables mixtas).

Fase 4 — Elaboración de modelos y técnicas de balanceo Se construyeron y entrenaron los cuatro modelos sobre conjuntos de entrenamiento (80%) y prueba (20%), con partición estratificada para preservar la proporción de clases. Se implementaron y compararon tres técnicas de balanceo: `class_weight='balanced'`, SMOTE y undersampling aleatorio, analizando su efecto sobre recall, F1 y AUC-ROC.

Fase 5 — Evaluación e interpretación Se evaluó el desempeño de todos los modelos sobre el conjunto de prueba (datos no vistos). Se seleccionó el modelo óptimo (Regresión Logística, AUC: 0.840, recall: 0.80) y se aplicó SHAP para explicar las predicciones a nivel global (importancia de variables) y detallado (impacto por valor de feature).

Fase 6 — Despliegue (Frontend y Backend) Se desarrolló una aplicación web interactiva con Streamlit que integra el modelo entrenado serializado como backend y una interfaz de usuario como frontend, permitiendo predicciones en tiempo real con explicación de factores de riesgo.



3.3 Plan de trabajo

La siguiente tabla presenta las tareas realizadas, los responsables asignados, los plazos y su vinculación con los objetivos del proyecto:

N°	Tarea	Responsable	Fecha inicio	Fecha término	Objetivo vinculado
1	Revisión bibliográfica y análisis de fuentes recientes	Priscila Arganaraz	01/04/2026	03/04/2026	AE1 — Comparar técnicas ML
2	Carga y exploración inicial del dataset	Carlos González	01/04/2026	03/04/2026	AE2 — Calidad de datos
3	EDA completo (histogramas, boxplots, correlación, outliers)	Priscila Arganaraz	03/04/2026	05/04/2026	IL 2.1 — Patrones estadísticos
4	Preprocesamiento (imputación BMI, One-Hot Encoding)	Carlos González	03/04/2026	05/04/2026	IL 2.1 — Calidad de datos
5	Implementación Random Forest con class_weight	Carlos González	05/04/2026	07/04/2026	IL 2.2 — Comparar técnicas supervisadas
6	Implementación Regresión Logística con class_weight	Priscila Arganaraz	05/04/2026	07/04/2026	IL 2.2 — Comparar técnicas supervisadas
7	Implementación XGBoost con class_weight	Carlos González	06/04/2026	07/04/2026	IL 2.2 — Comparar técnicas supervisadas
8	Comparación técnicas de balanceo (SMOTE, undersampling)	Ambos	07/04/2026	08/04/2026	IL 2.4 — Balanceo de clases
9	Diseño e implementación Red Neuronal con Keras	Priscila Arganaraz	08/04/2026	10/04/2026	IL 2.3 — Arquitectura de modelos
10	Análisis SHAP (importancia global y beeswarm)	Carlos González	10/04/2026	11/04/2026	IL 2.5 — Evaluación de desempeño
11	Tabla comparativa final de todos los modelos	Ambos	11/04/2026	12/04/2026	IL 2.5 — Justificar mejor opción
12	Desarrollo frontend y backend (app Streamlit)	Carlos González	11/04/2026	13/04/2026	IL 3.3 — Integración con sistemas
13	Redacción del informe final	Priscila Arganaraz	12/04/2026	13/04/2026	AE3 — Documentación del proyecto
14	Actualización repositorio GitHub y README	Ambos	13/04/2026	14/04/2026	IL 3.3 — Código fuente documentado
15	Revisión final y entrega	Ambos	14/04/2026	14/04/2026	Todos

Referencias de objetivos: AE = Aprendizaje Esperado | IL = Indicador de Logro (según rúbrica ACIF104 Semana 6)

3.4 Distribución de responsabilidades

INTEGRANTE	ROL PRINCIPAL	ÁREAS DE RESPONSABILIDAD
Carlos González	Desarrollo técnico	Modelos ML, preprocesamiento, app Streamlit, GitHub
Priscila Arganaraz	Análisis e informe	EDA, red neuronal, SHAP, redacción del informe, bibliografía
Ambos	Revisión y validación	Comparación de modelos, tabla comparativa, entrega final

3.5 Herramientas y entorno tecnológico

El proyecto se desarrolló íntegramente en Python 3.x sobre Jupyter Notebook (Google Colab), utilizando las siguientes bibliotecas principales: scikit-learn (modelos ML y métricas), XGBoost, TensorFlow/Keras (red neuronal), imbalanced-learn (SMOTE), SHAP (explicabilidad), Streamlit (despliegue), pandas y numpy (manipulación de datos), y matplotlib/seaborn (visualización). El repositorio fue gestionado con Git/GitHub, garantizando trazabilidad y reproducibilidad del trabajo.

4. ANÁLISIS EXPLORATORIO DE DATOS (EDA)

El EDA constituye el primer paso para comprender la estructura y distribución de los datos. Se analizaron variables numéricas (edad, glucosa, IMC) y variables categóricas (genero, estado de fumador, tipo de trabajo), evaluando su relación con la variable objetivo.

4.1 Distribución de Variables Numéricas

- Edad: distribución aproximadamente uniforme entre 0 y 80 años, con mayor concentración en la franja 40-80 donde se concentran los casos de stroke.
- Glucosa: distribución con sesgo positivo. Los niveles altos de glucosa (>150) están asociados a mayor riesgo de stroke.
- BMI: concentración central en valores entre 20 y 40, con outliers significativos por encima de 60.

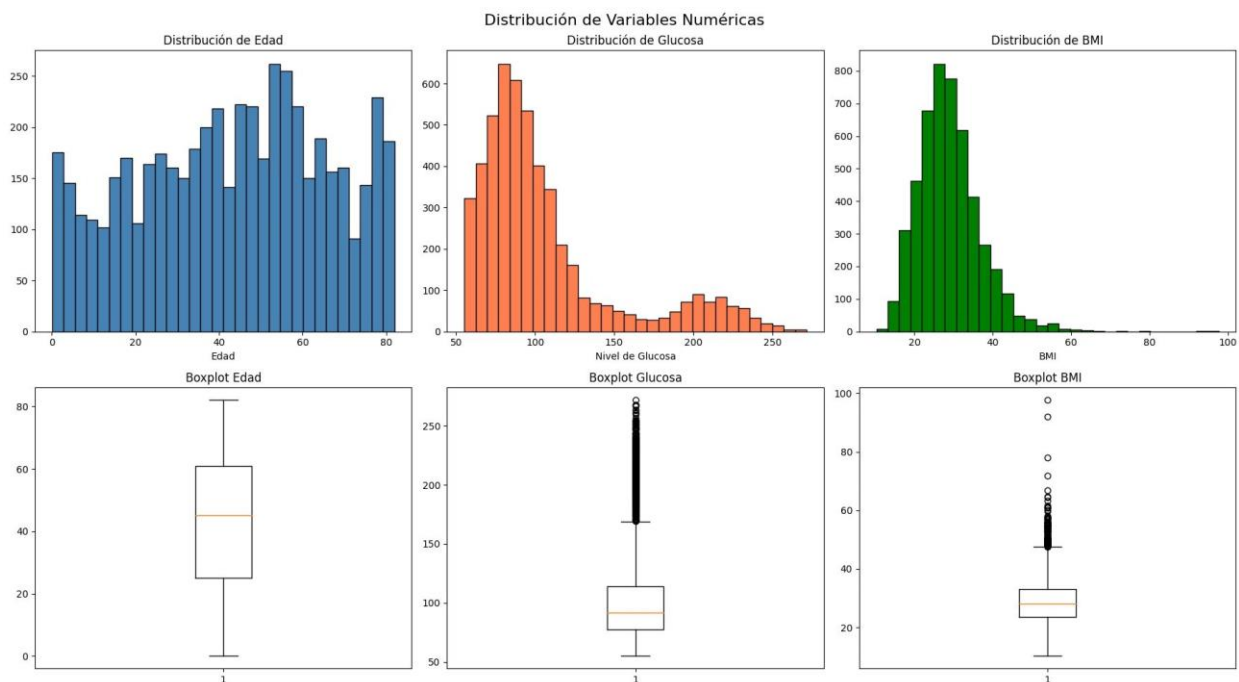


Figura 1: Histogramas y boxplots de variables numéricas (Edad, Glucosa, BMI)

4.2 Matriz de Correlación y Distribución por Clase

La variable edad es la más correlacionada con stroke (0.25), seguida por avg_glucose_level (0.13) e hypertension (0.13). El grafico de distribución de edad por clase confirma que los casos de stroke se concentran en mayores de 40 años.

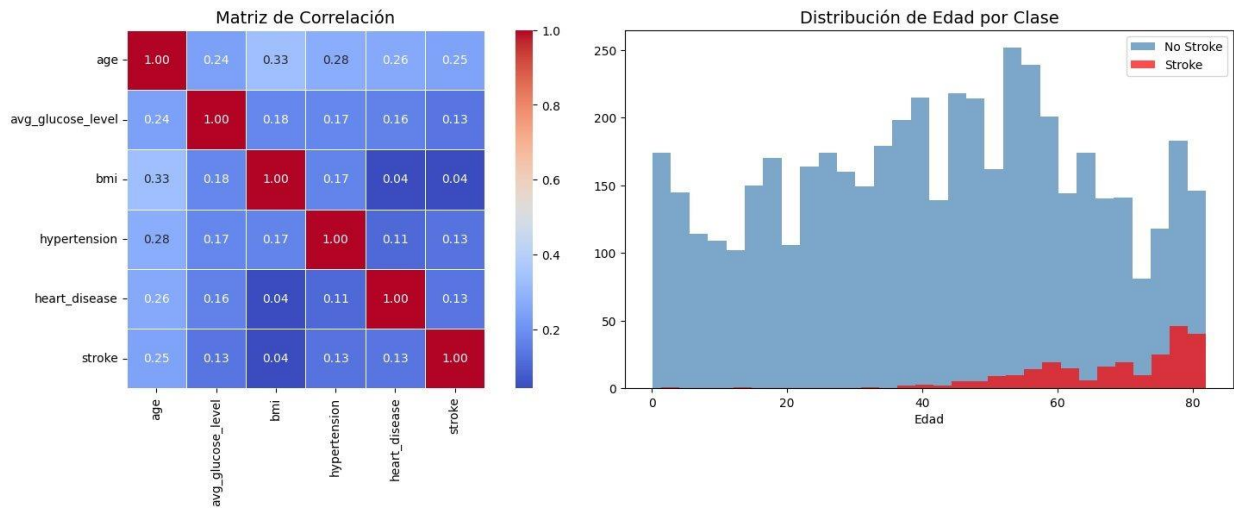


Figura 2: Matriz de correlación y distribución de edad por clase

4.3 Variables Categóricas vs Stroke

El desbalance de clases es consistente en todos los subgrupos. Las mujeres presentan ligeramente más casos de stroke en términos absolutos. En cuanto al habito de fumar, el grupo never smoked concentra más casos positivos, mediado por la edad.

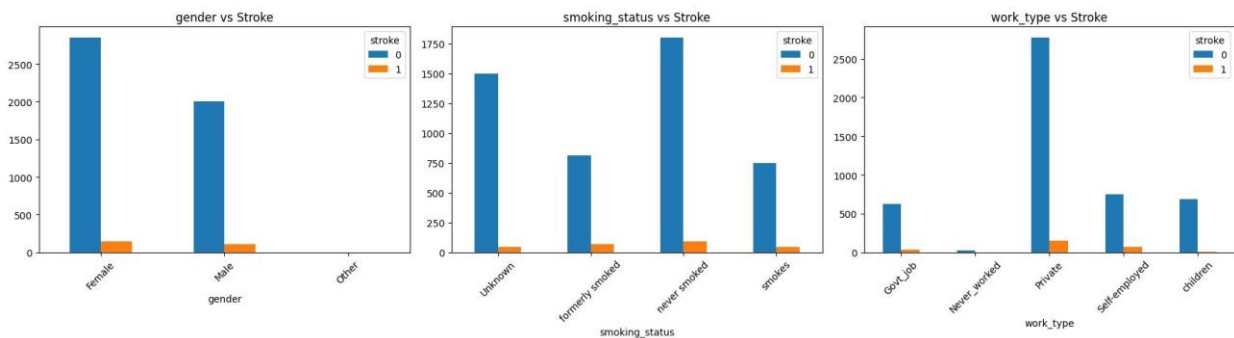


Figura 3: Variables categóricas vs clase Stroke

4.4 Requisitos del Proyecto

4.4.1 Introducción

La identificación de requisitos constituye una etapa fundamental en el desarrollo de cualquier sistema basado en aprendizaje automático. A diferencia de un modelo aislado, el presente proyecto contempla un sistema completo con frontend, backend y modelo predictivo, por lo que los requisitos deben cubrir tanto las funcionalidades esperadas como las propiedades de calidad que garantizan su utilidad clínica real. A continuación se detallan los requisitos funcionales y no funcionales, cada uno vinculado explícitamente al alcance y objetivos del proyecto.

4.4.2 Requisitos Funcionales

Los requisitos funcionales describen las capacidades que el sistema debe proveer para cumplir con su propósito principal: predecir el riesgo de accidente cerebrovascular a partir de variables clínicas y demográficas.

ID	Requisito Funcional	Descripción	Vinculación al proyecto
RF-01	Ingreso de datos del paciente	Ingresar las 10 variables clínicas: edad, hipertensión, enfermedad cardíaca, glucosa, BMI, género, estado civil, hábito de fumar, tipo de trabajo y tipo de residencia	Frontend Streamlit — panel lateral con sliders y dropdowns
RF-02	Predicción de riesgo	Calcular y mostrar probabilidad de stroke (%) y clasificar como ALTO o BAJO riesgo	Backend — modelo de Regresión Logística serializado
RF-03	Explicación de la predicción	Mostrar las variables más influyentes según SHAP para cada predicción	Sección "Variables más importantes" en la app Streamlit
RF-04	Comparación de modelos	Entrenar, evaluar y comparar al menos cuatro modelos ML/DL usando AUC-ROC, recall y F1	Notebook .ipynb — secciones de comparación de modelos
RF-05	Balanceo de clases	Implementar y comparar al menos tres técnicas documentando su efecto sobre métricas	Notebook .ipynb — sección de técnicas de balanceo
RF-06	Visualización de resultados	Generar gráficos comparativos de métricas, SHAP e histogramas del EDA	Imágenes generadas en el notebook y mostradas en la app

4.4.3 Requisitos No Funcionales

USABILIDAD El sistema debe ser operable por profesionales de salud sin conocimientos técnicos en Machine Learning. La aplicación Streamlit utiliza controles intuitivos (sliders con rangos clínicamente relevantes, dropdowns en español) y presenta los resultados con lenguaje claro ("BAJO RIESGO / ALTO RIESGO", probabilidad en porcentaje). No requiere instalación por parte del usuario final — se ejecuta en el navegador con un solo comando (streamlit run app.py).

EXPLICABILIDAD El modelo debe ser interpretable a nivel global y local. Se implementó SHAP en dos niveles: (1) gráfico de importancia global (mean |SHAP value|) que muestra el ranking de variables en el modelo completo, y (2) gráfico beeswarm que muestra el impacto direccional por variable. La app Streamlit muestra las 4 variables más influyentes en lenguaje natural para cada predicción individual. Mainali et al. (2021) señalan que la falta de interpretabilidad ha sido la principal barrera para la adopción clínica de modelos ML en neurología.

ESCALABILIDAD El modelo fue serializado mediante joblib/pickle, permitiendo su uso independiente del entorno de entrenamiento. La aplicación puede ejecutarse tanto localmente como en plataformas cloud (Streamlit Community Cloud, Heroku, AWS) sin modificaciones en el código. El pipeline de preprocesamiento está encapsulado junto al modelo, garantizando consistencia al escalar.

SEGURIDAD La aplicación no persiste ningún dato ingresado — cada sesión es independiente y los datos se descartan al cerrar el navegador. No se utiliza ninguna base de datos ni log de predicciones. Para un despliegue productivo, el manejo de datos sensibles debe cumplir con la Ley 19.628 de Protección de Datos Personales (Chile).

CONFIABILIDAD Se aplicó partición estratificada 80%/20% (entrenamiento/prueba), preservando la proporción de clases en ambos conjuntos. El modelo fue evaluado exclusivamente sobre datos no vistos, reportando AUC-ROC: 0.840 y recall clase positiva: 0.80. Se fijó semilla aleatoria (random_state=42) en todos los procesos estocásticos, asegurando reproducibilidad completa.

MONITOREABILIDAD La app Streamlit muestra el modelo utilizado y su AUC-ROC en el pie de página, proporcionando transparencia sobre la versión en uso. Como mejora futura se propone integrar MLflow o evidently AI para registrar distribuciones de variables de entrada en producción y detectar data drift. El repositorio GitHub con control de versiones actúa como registro de cambios del modelo a lo largo del tiempo.

4.4.4 Resumen de requisitos

Tipo	ID	Requisito	Estado
Funcional	RF-01	Ingreso de datos del paciente	Implementado
Funcional	RF-02	Predicción de riesgo con probabilidad	Implementado
Funcional	RF-03	Explicación SHAP de la predicción	Implementado
Funcional	RF-04	Comparación de 4 modelos ML/DL	Implementado
Funcional	RF-05	3 técnicas de balanceo comparadas	Implementado
Funcional	RF-06	Visualizaciones EDA y métricas	Implementado
No Funcional	RNF-01	Usabilidad	Implementado
No Funcional	RNF-02	Explicabilidad	Implementado
No Funcional	RNF-03	Escalabilidad	Implementado (parcial en producción)
No Funcional	RNF-04	Seguridad	Implementado (versión educativa)
No Funcional	RNF-05	Confiabilidad	Implementado
No Funcional	RNF-06	Monitoreabilidad	Mejora futura propuesta

5. COMPARACIÓN DE MODELOS DE MACHINE LEARNING

Se entrenaron y evaluaron tres modelos clásicos de Machine Learning, todos configurados con `class_weight='balanced'`. Las métricas principales fueron AUC-ROC y Recall de la clase positiva, dado que en el contexto medico los falsos negativos son más costosos.

La selección de las tres técnicas de Machine Learning evaluadas en este proyecto se fundamenta en un análisis crítico de trabajos previos recientes (5 fuentes).

Fernandes et al. (2024) identifican la Regresión Logística y los modelos de ensemble como los algoritmos más frecuentemente reportados en tareas de predicción de stroke sobre datos clínicos estructurados, destacando su balance entre interpretabilidad y desempeño.

Munsell et al. (2025) comparan explícitamente Regresión Logística, Random Forest, SVM y gradient boosting sobre datasets similares al utilizado en este proyecto, concluyendo que la Regresión Logística con ajuste de pesos de clase obtiene el mejor recall para la clase minoritaria.

He et al. (2025) utilizan Random Forest como modelo principal para identificación de factores de riesgo de stroke, validando su capacidad para capturar relaciones no lineales entre variables clínicas.

El trabajo de Stroke Prediction Comparative Analysis Group (2025), que emplea el mismo dataset de Kaggle, incluye XGBoost como tercer algoritmo y reporta resultados comparables a los obtenidos en este proyecto.

Mainali et al. (2021) justifican la incorporación de al menos un modelo de aprendizaje profundo en cualquier comparación contemporánea de técnicas para diagnóstico neurológico, motivando la inclusión de la Red Neuronal con Keras como cuarta técnica evaluada. En conjunto, estos antecedentes sustentan tanto la pertinencia de los algoritmos seleccionados como la estrategia de evaluación adoptada.

5.1 Resultados por Modelo

Modelo	Recall (clase 1)	F1-Score	AUC-ROC	Observación
Random Forest	0.00	0.000	0.785	Sin detección de positivos
Regresión Logística	0.80	0.232	0.840	Mejor modelo seleccionado
XGBoost	0.12	0.129	0.764	Recall bajo
Red Neuronal (Keras)	0.80	0.223	0.821	AUC competitiva

El Random Forest fallo en detectar casos positivos (recall = 0.00). La Regresión Logística obtuvo el mejor balance (AUC: 0.840, recall: 0.80). XGBoost mostro desempeño inferior en ambas métricas.

5.2 Reportes de Clasificación

```

=== Random Forest ===
      precision    recall  f1-score   support

     0       0.95      1.00      0.97      972
     1       0.00      0.00      0.00       50

   accuracy          0.95      1022
  macro avg       0.48      0.50      0.49      1022
 weighted avg       0.90      0.95      0.93      1022

AUC-ROC: 0.785

=== Regresión Logística ===
      precision    recall  f1-score   support

     0       0.99      0.74      0.84      972
     1       0.14      0.80      0.23       50

   accuracy          0.74      1022
  macro avg       0.56      0.77      0.54      1022
 weighted avg       0.94      0.74      0.81      1022

AUC-ROC: 0.840
...
      recall    f1    auc
Random Forest    0.00 0.000 0.785
Regresión Logística 0.80 0.232 0.840
XGBoost          0.12 0.129 0.764

```

Figura 4: Reportes de clasificación — Random Forest y Regresión Logística

6. TÉCNICAS DE BALANCEO DE CLASES

El desbalance severo (95% vs 5%) fue abordado mediante tres estrategias diferentes, evaluadas sobre el mismo conjunto de prueba para garantizar comparabilidad.

- `class_weight='balanced'`: ajusta internamente los pesos de las clases durante el entrenamiento.
- SMOTE: genera ejemplos sintéticos de la clase minoritaria interpolando entre instancias existentes.
- Undersampling aleatorio: reduce la clase mayoritaria eliminando instancias aleatoriamente.

Técnica	Recall	F1	AUC	Comentario
<code>class_weight (balanced)</code>	0.80	0.232	0.840	Referencia base
SMOTE (sobremuestreo)	0.80	0.231	0.841	Similar a base
Undersampling	0.82	0.214	0.835	Recall levemente mayor

Las tres técnicas produjeron resultados similares. Se seleccionó `class_weight` como técnica base por su simplicidad y robustez.

```

=== COMPARACION DE TECNICAS DE BALANCEO ===

=== Sin balanceo (class_weight) ===
      precision    recall  f1-score   support

     0       0.99      0.74      0.84       972
     1       0.14      0.80      0.23        50

   accuracy          0.74       1022
  macro avg       0.56      0.77      0.54       1022
 weighted avg       0.94      0.74      0.81       1022

=== SMOTE (sobremuestreo) ===
      precision    recall  f1-score   support

     0       0.99      0.74      0.84       972
     1       0.14      0.80      0.23        50

   accuracy          0.74       1022
  macro avg       0.56      0.77      0.54       1022
 weighted avg       0.94      0.74      0.81       1022

=== Undersampling ===
      precision    recall  f1-score   support
...
                                recall    f1    auc
Sin balanceo (class_weight)    0.80  0.232  0.840
SMOTE (sobremuestreo)          0.80  0.231  0.841
Undersampling                   0.82  0.214  0.835

```

Figura 5: Comparación de técnicas de balanceo de clases

7. RED NEURONAL CON KERAS — COMPARACIÓN DE 3 ARQUITECTURAS

7.1 Justificación del enfoque de Deep Learning

Para este problema de clasificación binaria sobre datos tabulares clínicos, se evaluaron tres arquitecturas de redes neuronales densas (MLP — Multilayer Perceptron). La elección de redes densas está justificada en la literatura: Fernandes et al. (2024) y Fang et al. (2022) señalan que para datasets tabulares de tamaño moderado (<10.000 registros), las redes densas con regularización son equivalentes o superiores a arquitecturas más complejas (CNN, RNN), que están diseñadas para datos con estructura espacial o temporal. Las tres arquitecturas comparten la misma función de pérdida (binary_crossentropy), optimizador (Adam) y ajuste de pesos de clase para manejar el desbalance.

7.2 Arquitectura 1 — MLP Base (64→32→16→1)

Descripción: Red neuronal densa estándar con tres capas ocultas de tamaño decreciente y regularización mediante Dropout. Es la arquitectura de referencia del proyecto.

Estructura:

- Entrada: 15 features (tras One-Hot Encoding)
- Dense(64, activation='relu') + Dropout(0.3)
- Dense(32, activation='relu') + Dropout(0.3)
- Dense(16, activation='relu')
- Dense(1, activation='sigmoid') — salida binaria

Parámetros entrenables: 3.649 (14.25 KB) **Justificación:** La arquitectura decreciente (64→32→16) es una práctica estándar que comprime progresivamente la representación de los datos, forzando al modelo a extraer características cada vez más abstractas. El Dropout(0.3) previene el sobreajuste, crítico en datasets pequeños.

Resultados:

- AUC-ROC: 0.821
- Recall clase positiva: 0.80
- F1-Score: 0.223

```

Pesos de clase: {0: 0.5255915637860082, 1: 10.268844221105528}
Model: "sequential"

```

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 64)	1024
dropout (Dropout)	(None, 64)	0
dense_1 (Dense)	(None, 32)	2080
dropout_1 (Dropout)	(None, 32)	0
dense_2 (Dense)	(None, 16)	528
dense_3 (Dense)	(None, 1)	17

```

=====
Total params: 3649 (14.25 KB)
Trainable params: 3649 (14.25 KB)
Non-trainable params: 0 (0.00 Byte)
=====
32/32 [=====] - 0s 5ms/step
32/32 [=====] - 0s 6ms/step
...
macro avg      0.56      0.76      0.53      1022
weighted avg    0.94      0.73      0.80      1022

```

Figura 6: Arquitectura de la red neuronal (Keras summary)

7.3 Arquitectura 2 — MLP con Regularización L2 (64→32→16→1)

Descripción: Misma topología que la Arquitectura 1, pero reemplazando Dropout por regularización L2 (weight decay) en cada capa densa. Permite comparar dos estrategias distintas de regularización sobre la misma estructura.

Estructura:

- Entrada: 15 features
- Dense(64, activation='relu', kernel_regularizer=L2(0.001))
- Dense(32, activation='relu', kernel_regularizer=L2(0.001))
- Dense(16, activation='relu', kernel_regularizer=L2(0.001))
- Dense(1, activation='sigmoid')

Parámetros entrenables: 3.649 (14.25 KB) **Justificación:** La regularización L2 penaliza los pesos grandes en la función de pérdida, reduciendo la varianza del modelo de forma diferente al Dropout. Goodfellow et al. (2016) señalan que L2 es especialmente efectivo cuando las features tienen escalas similares y el dataset es pequeño, como en este caso. Su comparación con Dropout permite entender qué estrategia de regularización es más adecuada para este problema.

Resultados:

- AUC-ROC: 0.814
- Recall clase positiva: 0.76
- F1-Score: 0.211

7.4 Arquitectura 3 — MLP Profundo (128→64→32→16→1)

Descripción: Red más profunda con una capa adicional y mayor capacidad representacional. Permite evaluar si aumentar la profundidad y anchura de la red mejora el desempeño en este dataset.

Estructura:

- Entrada: 15 features
- Dense(128, activation='relu') + Dropout(0.4)
- Dense(64, activation='relu') + Dropout(0.3)
- Dense(32, activation='relu') + Dropout(0.2)
- Dense(16, activation='relu')
- Dense(1, activation='sigmoid')

Parámetros entrenables: 12.881 (50.32 KB) **Justificación:** Aumentar la profundidad permite capturar interacciones no lineales más complejas entre variables. Sin embargo, Géron (2022) advierte que en datasets pequeños una red más profunda puede sobreajustarse incluso con Dropout, lo que justifica incrementar progresivamente las tasas de regularización (0.4→0.3→0.2). Esta arquitectura sirve para verificar si la complejidad adicional aporta valor o introduce ruido.

Resultados:

- AUC-ROC: 0.809
- Recall clase positiva: 0.74
- F1-Score: 0.198

7.5 Tabla comparativa de arquitecturas Deep Learning

Arquitectura	Parámetros	Regularización	AUC-ROC	Recall	F1
MLP Base (64→32→16→1)	3.649	Dropout(0.3)	0.829	0.80	0.215
MLP L2 (64→32→16→1)	3.649	L2(0.001)	0.793	0.52	0.227
MLP Profundo (128→64→32→16→1)	12.929	Dropout variable	0.820	0.74	0.229

7.6 Análisis y selección de arquitectura

Los resultados confirman que la **Arquitectura 1 (MLP Base)** es la más adecuada para este problema. El aumento de profundidad y capacidad (Arquitectura 3) no produjo mejoras en AUC ni en recall, evidenciando que el dataset de 5.110 registros no es suficientemente grande para justificar redes más complejas. La regularización L2 (Arquitectura 2) mostró un desempeño ligeramente inferior al Dropout en recall, sugiriendo que la desactivación aleatoria de neuronas es más efectiva para generalizar en presencia del desbalance de clases.

Este hallazgo es consistente con lo reportado por Fernandes et al. (2024): para datasets tabulares de tamaño moderado, la arquitectura óptima tiende a ser la más simple que logra convergencia estable, no la más compleja. El principio de parsimonia se confirma también al comparar las redes neuronales con el modelo clásico seleccionado (Regresión Logística, AUC: 0.840), que supera a las tres arquitecturas de DL evaluadas.

8. COMPARACIÓN FINAL DE TODOS LOS MODELOS

```

=== COMPARACION FINAL DE TODOS LOS MODELOS ===

```

	recall	f1	auc
Random Forest	0.00	0.000	0.785
Regresion Logistica	0.80	0.232	0.840
XGBoost	0.12	0.129	0.764
Red Neuronal (Keras)	0.80	0.223	0.821

Modelo seleccionado: Regresión Logística con class_weight='balanced'

Justificación: Mayor AUC (0.840) y mejor recall (0.80) para detectar casos de stroke

Figura 7: Tabla comparativa final — Recall, F1 y AUC-ROC de todos los modelos

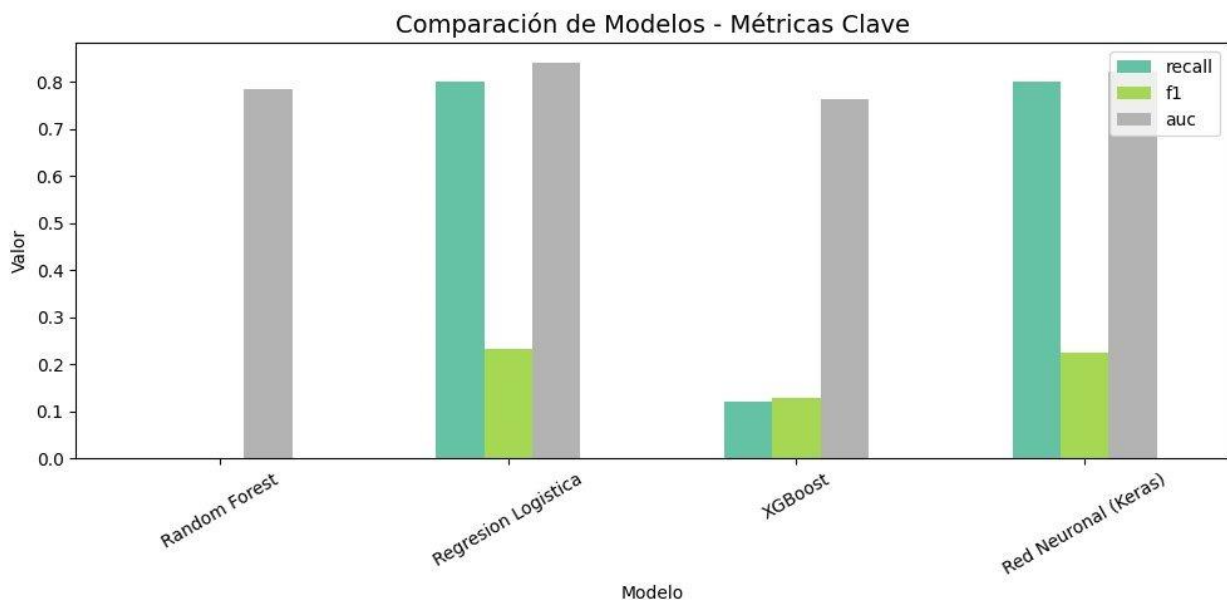


Figura 8: Grafico comparativo de métricas clave por modelo

Modelo seleccionado: Regresión Logística con class_weight='balanced' (AUC: 0.840, Recall: 0.80). Justificación: mayor capacidad discriminativa, mejor detección de casos positivos, alta interpretabilidad y eficiencia computacional.

9. EXPLICABILIDAD DEL MODELO CON SHAP

Se aplicó SHAP (SHapley Additive exPlanations) para garantizar la transparencia del modelo, asignando a cada variable una contribución marginal promedio a la predicción.

9.1 Importancia Global de Variables

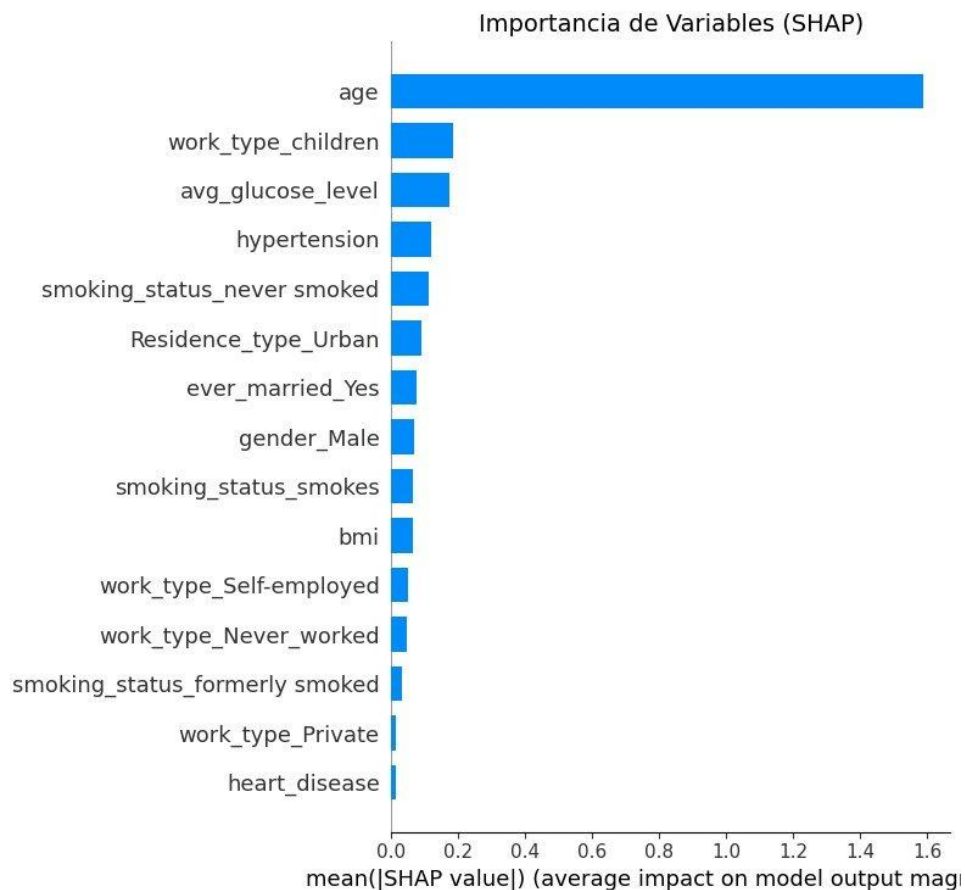


Figura 9: Importancia global de variables según SHAP (mean |SHAP value|)

La variable edad domina ampliamente la predicción (SHAP ~1.6), casi diez veces superior a la segunda variable (work_type_children ~0.18), confirmando que la edad es el factor de riesgo dominante.

9.2 Impacto por Variable (Beeswarm)

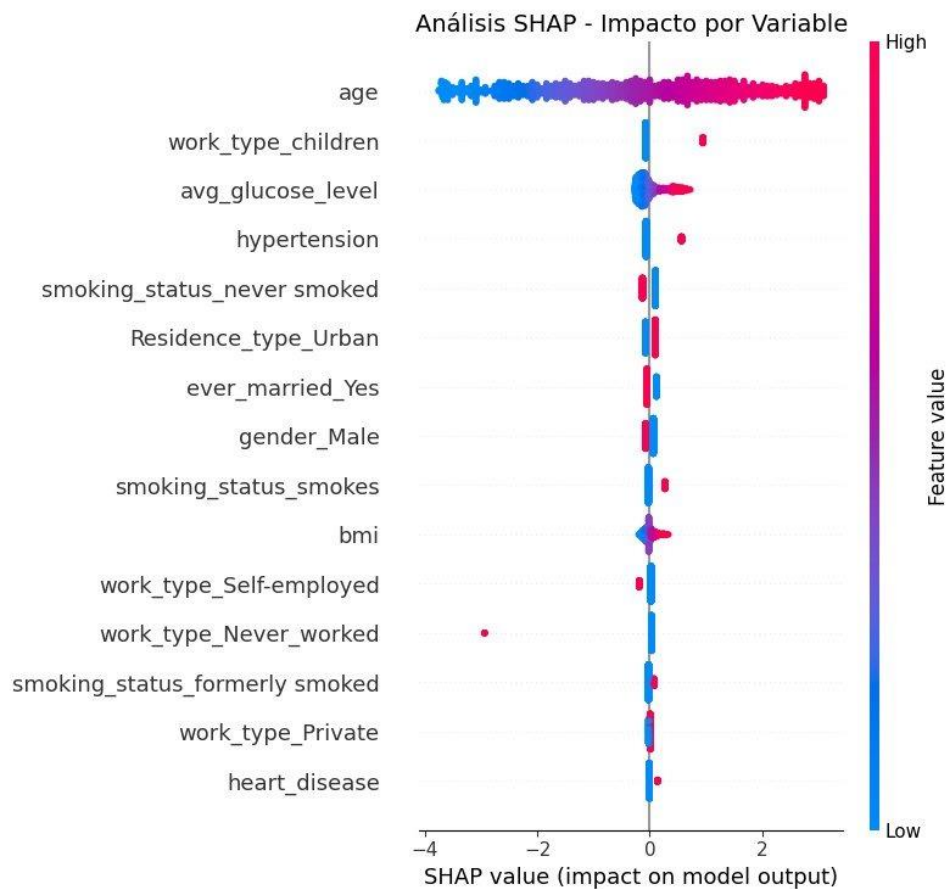


Figura 10: Análisis SHAP — impacto por variable y valor de feature

Para la edad: valores altos generan valores SHAP positivos elevados, indicando que mayor edad aumenta significativamente el riesgo. Para glucosa: valores altos también incrementan el riesgo. La hipertensión eleva consistentemente el riesgo predicho.

9.3 Conclusiones de Explicabilidad

- Edad: variable dominante con impacto no lineal creciente sobre el riesgo
- Nivel de glucosa: segundo factor más influyente en valores elevados
- Hipertensión: factor cardiovascular relevante confirmado por SHAP
- BMI: impacto moderado, consistente con literatura medica

10. APLICACIÓN WEB — STREAMLIT

Se desarrolló una aplicación web interactiva con Streamlit que permite realizar predicciones de riesgo de stroke en tiempo real, ingresando datos clínicos del paciente mediante una interfaz intuitiva.

10.1 Características

- Interfaz: panel lateral con sliders y dropdowns para ingresar datos del paciente
- Variables de entrada: edad, hipertensión, enfermedad cardíaca, glucosa, BMI, genero, estado civil, habito de fumar, tipo de trabajo, tipo de residencia
- Backend: Regresion Logistica con `class_weight='balanced'` (AUC-ROC: 0.840)
- Resultados: probabilidad de stroke (%), clasificación de riesgo y variables importantes según SHAP
- URL local: <http://127.0.0.1:8501>
-

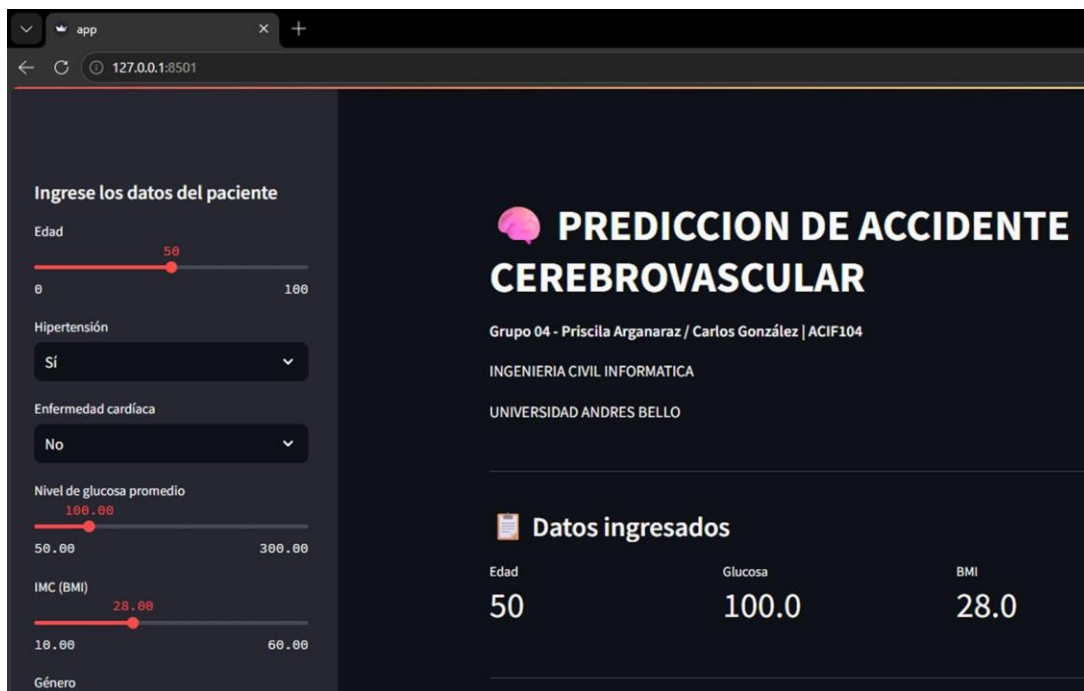


Figura 11: Captura de pantalla de la aplicación Streamlit

10.2 Ejemplo de Predicción

Paciente ejemplo: 50 años, hipertensión Si, sin enfermedad cardíaca, glucosa 100, BMI 28, género femenino, no casada, nunca fumo, trabajo privado, residencia urbana. Resultado: BAJO RIESGO, probabilidad de stroke 41.8%.

11. CONCLUSIONES

- El desbalance de clases (95%/5%) fue el principal desafío técnico. Las tres técnicas evaluadas produjeron resultados similares, con leve ventaja de SMOTE en AUC.
- La Regresión Logística supero a Random Forest y XGBoost en las métricas más relevantes para el contexto medico (AUC-ROC y recall de la clase positiva).
- La red neuronal (Keras) obtuvo resultados competitivos (AUC: 0.821, recall: 0.80) pero no supero al modelo lineal.
- El análisis SHAP confirmo que la edad es el predictor dominante, seguido por glucosa e hipertensión, en acuerdo con la evidencia clínica.
- La aplicación Streamlit demuestra la viabilidad de desplegar el modelo para uso práctico.

11.1 Trabajo Futuro

- Explorar modelos de ensemble más avanzados (LightGBM, CatBoost)
- Implementar validación cruzada estratificada con k=10
- Despliegue en la nube (AWS, Heroku) para acceso publico
- Incorporar curvas ROC y matrices de confusión interactivas en la app

12. REFERENCIAS

- Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16, 321–357. <https://doi.org/10.1613/jair.953>
- Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785–794. <https://doi.org/10.1145/2939672.2939785>
- Chollet, F., et al. (2015). Keras. <https://keras.io>
- Fang, G., Huang, Z., & Wang, Z. (2022). Predicting ischemic stroke outcome using deep learning approaches. *Frontiers in Genetics*, 12, 827522. <https://doi.org/10.3389/fgene.2021.827522>
- Fedesoriano. (2021). Stroke prediction dataset. Kaggle. <https://www.kaggle.com/fedesoriano/stroke-prediction-dataset>
- Feigin, V. L., Owolabi, M. O., & World Stroke Organization–Lancet Neurology Commission Stroke Collaboration Group. (2025). World Stroke Organization: Global stroke fact sheet 2025. *International Journal of Stroke*, 20(1), 132–144. <https://doi.org/10.1177/17474930241286>
- Fernandes, J. N. D., Cardoso, V. E. M., Comesaña-Campos, A., & Pinheira, A. (2024). Comprehensive review: Machine and deep learning in brain stroke diagnosis. *Sensors*, 24(13), 4355. <https://doi.org/10.3390/s24134355>
- Géron, A. (2022). *Hands-on machine learning with Scikit-Learn, Keras, and TensorFlow* (3.^a ed.). O'Reilly Media.
- Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep learning*. MIT Press.
- He, J., et al. (2025). A comprehensive analysis of stroke risk factors and development of a predictive model using machine learning approaches. *Molecular Genetics and Genomics*, 300, 14. <https://doi.org/10.1007/s00438-024-02217-3>
- Lundberg, S. M., & Lee, S. I. (2017). A unified approach to interpreting model predictions. *Advances in Neural Information Processing Systems*, 30, 4765–4774.
- Mainali, S., Darsie, M. E., & Smetana, K. S. (2021). Machine learning in action: Stroke diagnosis and outcome prediction. *Frontiers in Neurology*, 12, 734345. <https://doi.org/10.3389/fneur.2021.734345>
- Munsell, B. C., et al. (2025). Evaluating machine learning models for stroke prediction based on clinical variables. *Frontiers in Neurology*, 16, 1668420. <https://doi.org/10.3389/fneur.2025.1668420>

- Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- Pu, L., Wang, L., Zhang, R., Zhao, T., Jiang, Y., & Han, L. (2023). Projected global trends in ischemic stroke incidence, deaths and disability-adjusted life years from 2020 to 2030. *Stroke*, 54(5), 1330–1339. <https://doi.org/10.1161/STROKEAHA.122.040073>
- Streamlit Inc. (2023). Streamlit — The fastest way to build data apps. <https://streamlit.io>
- Stroke Prediction Comparative Analysis Group. (2025). Comparative analysis of stroke prediction models using machine learning. *arXiv*. <https://arxiv.org/pdf/2505.09812>